Computer Organization

Lab1:Introduction

Rars, Vivado

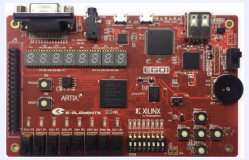
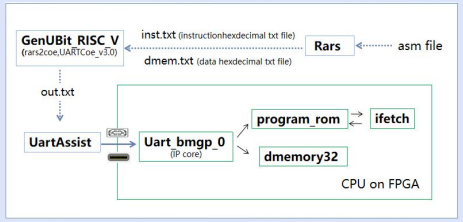
Tool kits(1)

Topic

- **laboratory course Goals, Tasks and Rules.**
- **Tool kits(1)**
  - **Rars**
    - ✓ related to Computer Organization
    - ✓ practice on **Rars** and a very basic part of **RISC-V**
  - **Vivado**
    - ✓ related to Digital Design(fundamental of Computer Organization)
    - ✓ practice on **Vivado** and **Verilog**

Laboratory course Goals, Tasks and Tool kits

laboratory course **Goals**: hardware-software co-design

Task	Tool kits	Language
Learn and practice **RISC-V** (a type of Assemblly language)	➢ **Rars (rars_27a7c1f)**	RISC-V (software)
Design and implement an **CPU** (a special type of hardware)	➢ **Vivado** ➢ **FPGA based Development Board** EGO1 	Verilog (hardware) Tcl [optional] (Vivado constraints file)
Test the **CPU** with **program(s)** based on **RISC-V**	➢ **Assembler (Rars)** ➢ **Uart Tools (GenUBit_RISC_V, UartAssist)** ➢ **Vivado** ➢ **FPGA based Development Board(EGO1)** 	RISC-V Verilog Tcl [optional]

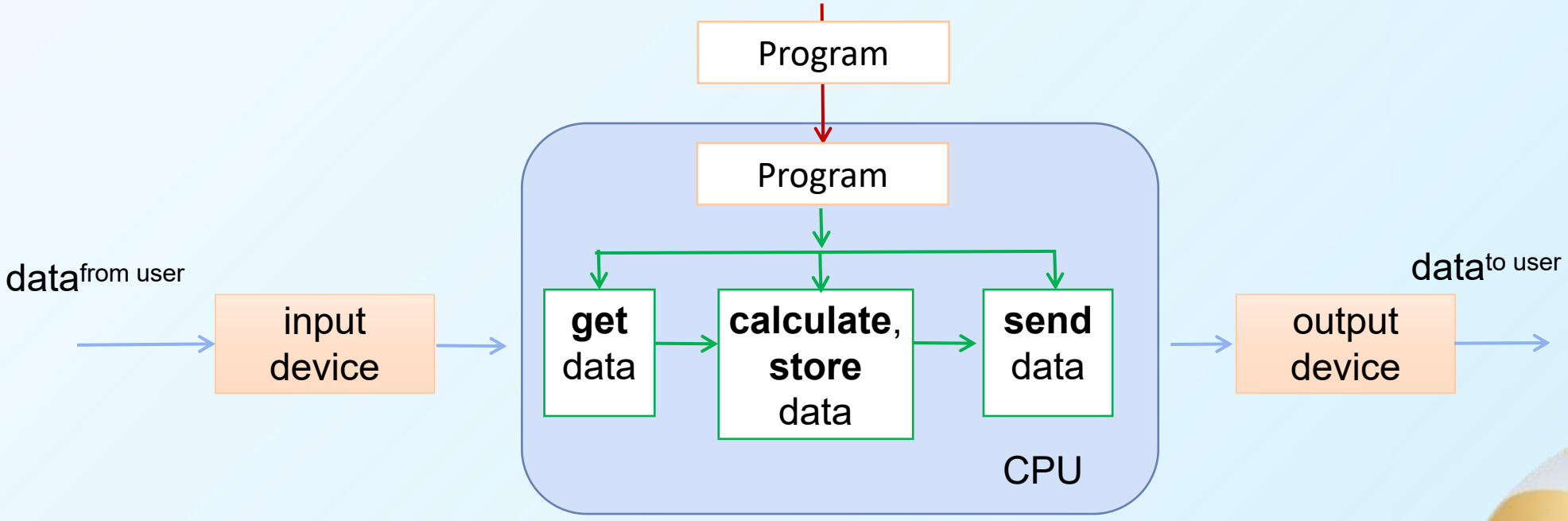
Assignments Submission Rules

- All assignments **MUST be submitted to BlackBoard site or 头歌 before the DDL. Any form of delayed submission is not accepted, and the score for delayed submission assignments is 0**.
- In the case of plagiarism: at the 1st time, the assignment was 0 for all concerned students and at the 2nd time, the grade of the experimental course is 0 for all concerned students.
- Reminder:
  - Assignment scoring and the score publication would be completed within two weeks after the DDL of assignment. **If you have any question about the score, please email the relevant reviewer in one week after the score publication.**

CPU + Program

➢ CPU work with Program

- CPU: **C**entral **P**rocessing **U**nit
- load Program 1st, execution it 2nd
- Process (**get**, **calculate**, **store**, **send**) specified data according to instructions



Software Programming

High level language vs Low level language

- High level language：
  - ✓ Don’t consider the hardware(Python, Java)
  - ✓ Focus on hardware but not much(C)

```
C:\Users\sustech>python
Python 3.11.4 (tags/v3.11.
Type "help", "copyright",
>>> a=input()
to CS202
>>> b="Welcome "
>>> print(b+a)
Welcome to CS202
>>>
```

Program
- data*
- instruction

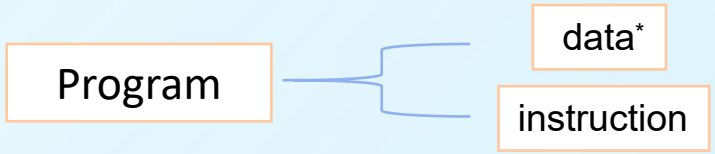
- **Low level** language:
  - ✓ Closely related to hardware
  - ✓ **Assembly language: RISC-V**, MIPS, ARM, x86.

```
# Demo
.data
        str:          .ascii   "Welcome "
        startpoint:   .space  9

.text
main:
        li a7,8       # to get a string
        la a0,startpoint
        li a1,10
        ecall

        li a7,4       # to print a string
        la a0,str
        ecall

        li a7,10      # to exit
        ecall
```

Assembly Program Structure

Data declarations + Program code + Comments(optional but suggested)

- **Part1: Data Declarations**
  - placed in section of program identified with assembler directive(汇编说明/汇编器指示符): **.data**
  - **declare** variable names used in program; storage allocated in main **memory** (RAM)



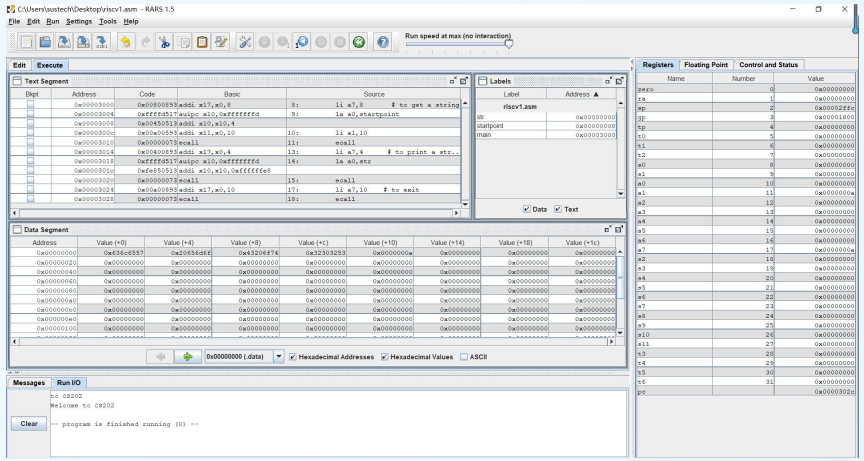
- **Part2: Program Code / Instruction**
  - placed in section of text identified with assembler directive: **.text**
  - contains program code (instructions)
  - starting point of code, e.g. ecution given label main.

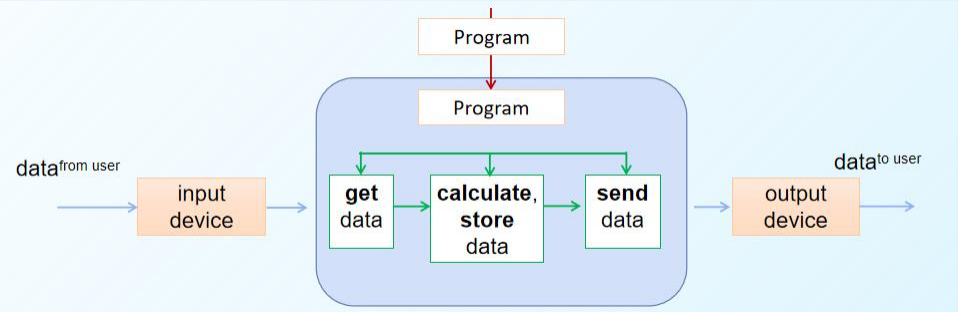
```
.data
        # here data* is 1byte, its initial value is 8'b0000_0001
        buf :  .byte  0x01

.text                   # instructions
main:
        lw  x1,  0(x0)
        sw x1,  4(x31)

        # jump to the instructions labled by main
        j main
```

- **Part3: Comments (suggested)**
  - anything following # on a line
    # This stuff would be considered as comment

Tool kits(1) - RARS

- **Rars: a lightweight assembly and runtime simulation tool specifically designed for the RISC-V.**
  - It integrates **assembly**, disassembly, **simulated execution**, and **debugging** functions, and provides highly visualized monitoring of runtime **memory** and **register** status, greatly reducing the learning and debugging threshold for RISC-V assembly language.



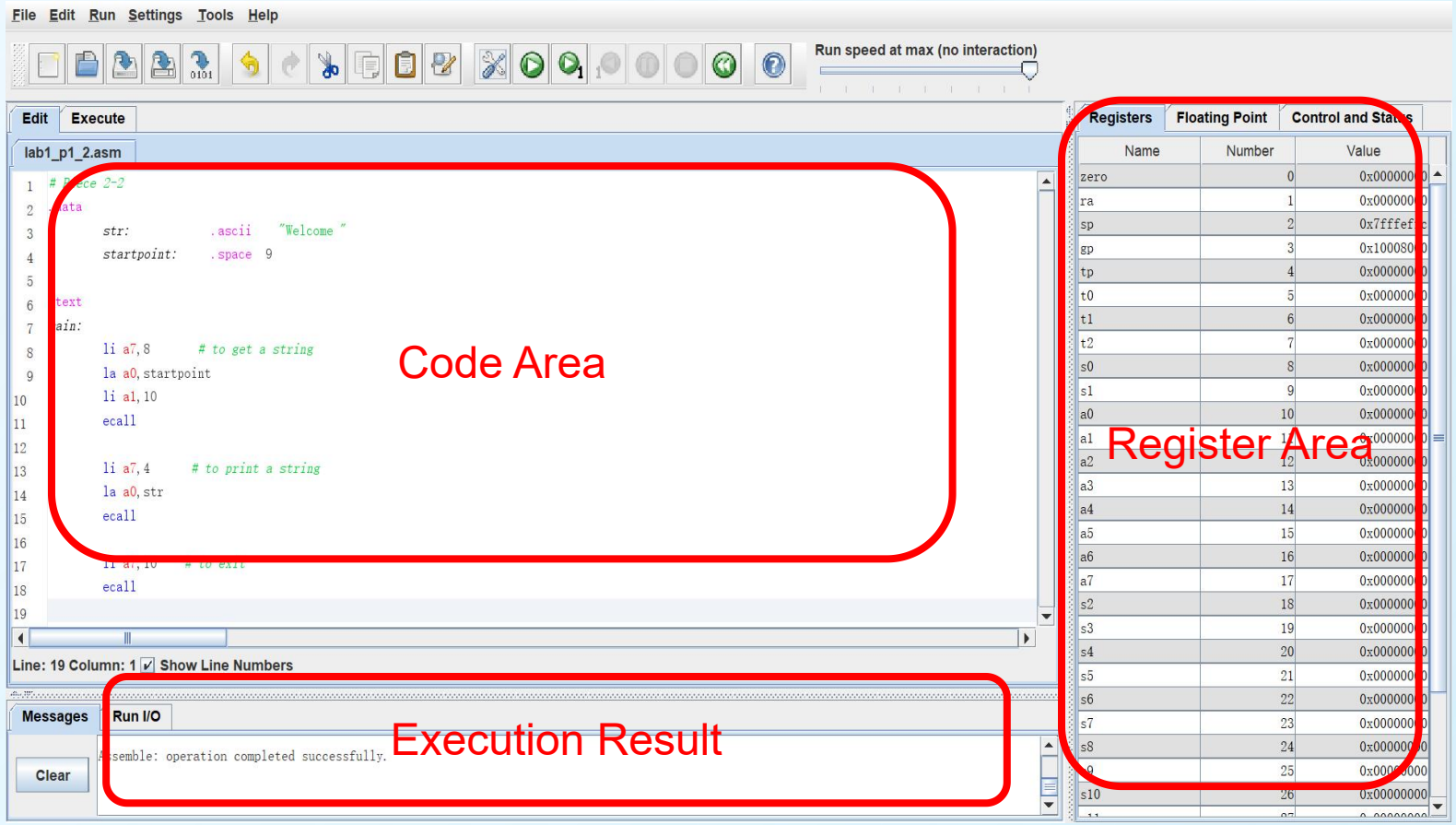


- **Download**
  - https://github.com/TheThirdOne/rars/releases

- **Run (Plug and play, no installation required:** double-click to run)
  - **NOTES: The operation of Rars requires the support of Java 8 or a later version.**

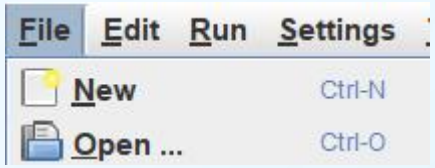
RARS - Quick Start(1)

- **Code area** is reserved for RISC-V assembly program editing.
- **Register area** displays latest register content.
  - In RISC-V, Arithmetic operands are in registers.
- **Execution area** displays the execution result.
  - “Messages” display the system information.
  - “Run I/O” used for input and output.



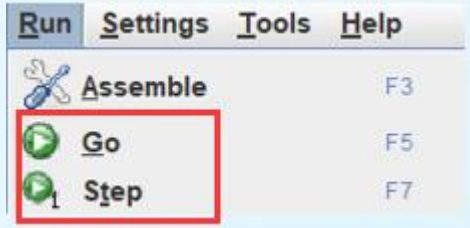
RARS - Quick Start(2)

- **Step1. Open/New**
  - ✓ option1. Open an exist assembly source file, then edit and save it.
  - ✓ option2. New an assembly source file,then edit and save it.

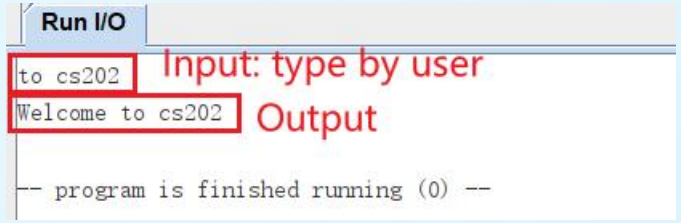


- **Step2. Assemble**

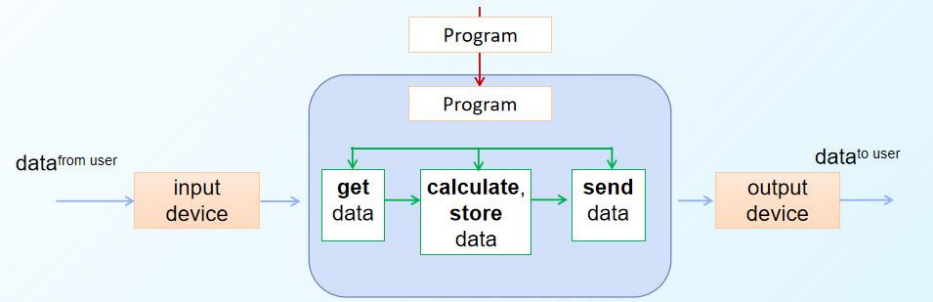
- **Step3. Run**
  - ✓ option1. Run the current program.
  - ✓ option2. Run step by step(for debugging)



- **Step4. Check for the result**
  - in sub-window of Rars: Run I/O




```
# Demo1
.data
        str:            .ascii    "Welcome "
        startpoint:     .space    9

.text
main:
        li a7,8         # to get a string
        la a0,startpoint
        li a1,10
        ecall

        li a7,4         # to print a string
        la a0,str
        ecall

        li a7,10        # to exit
        ecall
```



RARS - System Call

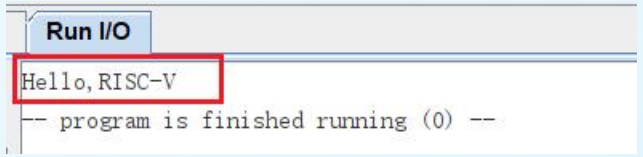
- A number of system services, mainly for input and output, are available in Rars.
- Example: display a string on the console(a7 = 4) and exit the program(a7 = 10).

```
# Demo2
.data
str: .asciz "Hello,RISC-V"

.text
li a7,4
la a0,str
ecall

li a7,10
ecall
```

Table of Available Services

Name	Number	Description	Inputs	Ouputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintFloat	2	Prints a floating point number	fa0 = float to print	N/A
PrintDouble	3	Prints a double precision floating point number	fa0 = double to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Reads an int from input console	N/A	a0 = the int
ReadFloat	6	Reads a float from input console	N/A	fa0 = the float
ReadDouble	7	Reads a double from input console	N/A	fa0 = the double
ReadString	8	Reads a string from the console	a0 = address of input buffer a1 = maximum number of characters to read	N/A
Sbrk	9	Allocate heap memory	a0 = amount of memory in bytes	a0 = address to the allocated block
Exit	10	Exits the program with code 0	N/A	N/A
PrintChar	11	Prints an ascii character	a0 = character to print (only lowest byte is considered)	N/A
ReadChar	12	Reads a character from input console	N/A	a0 = the character

Show output on bottom of the console:



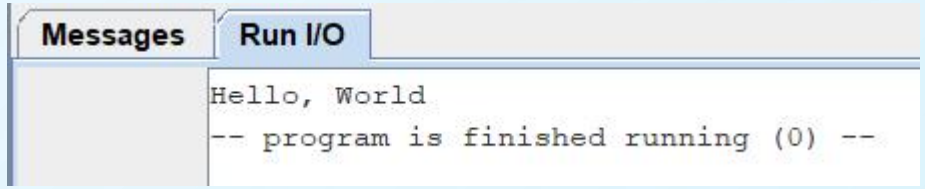
Tip: display all the system services information in "Help" of Rars.

Practice1

Choose one of the two tasks(1-1, 1-2) to complete.

- 1-1. Modify(if needed) the code in Demo1 to achieve the following functions: print out: "Hello, World".

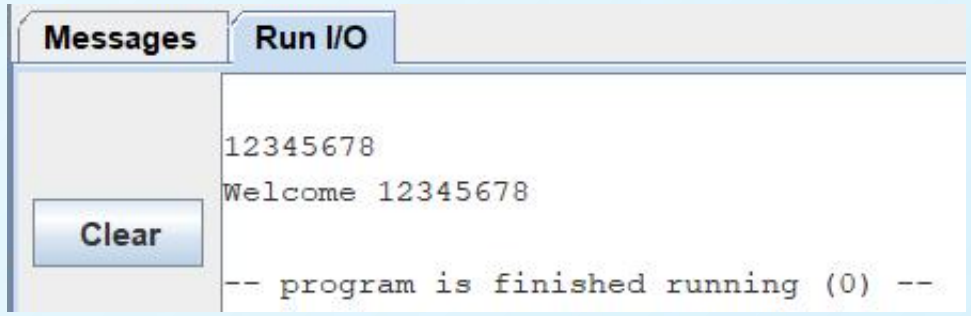
```
# Demo1
.data
str: .asciz "Hello,RISC-V"

.text
li a7,4
la a0,str
ecall

li a7,10
ecall
```



Practice1

- 1-2. Modify(if needed) the code in Demo2 to achieve the following functions:
  - get the input string(the total number of characters in the string does not exceed 8)
  - Concatenate the input string with the string "Welcome" (with "Welcome" coming before the input string) and output the result.

```
# Demo2
.data
        str:            .asciz    "Welcome "
        startpoint:   .space  9

.text
main:
        li a7,8       # to get a string
        la a0,startpoint
        li a1,10
        ecall

        li a7,4       # to print a string
        la a0,str
        ecall

        li a7,10    # to exit
        ecall
```



Recall Vivado and Verilog

Digital circuit system development process:

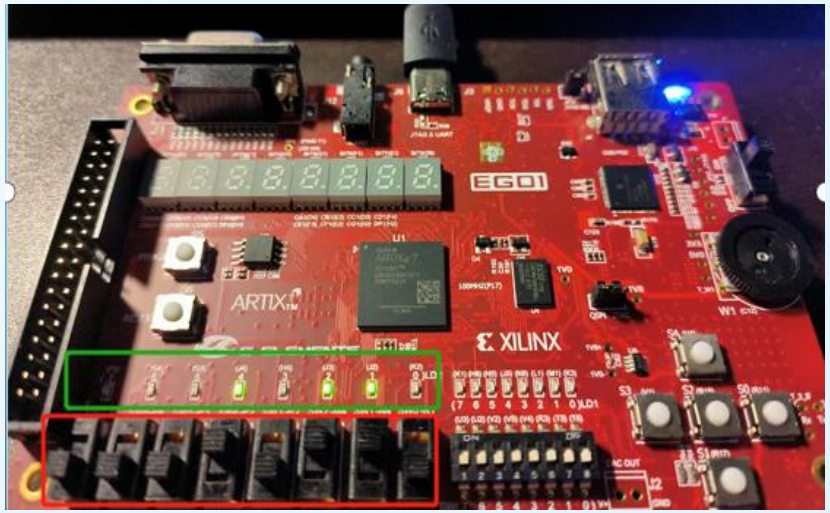
Design file(in verilog) + Constraint file

----(Vivado generate bitstream)---> **bitstream** file

---(Vivado hw manager, connect, program device)

---> the circuit is implemented on the **FPGA chip**

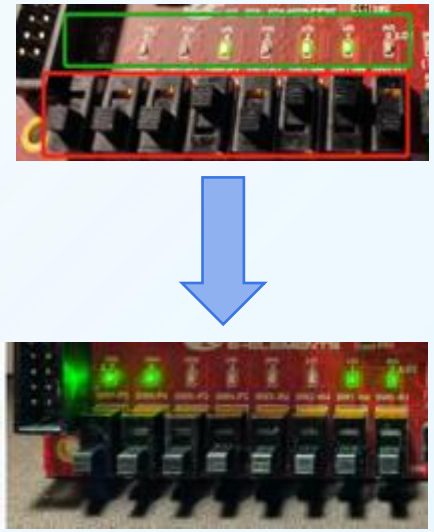
---> **Test** the circuit on the **FPGA based Development Board**




```
//Design file by verilog, save as sw2led.v
module sw2led(sw,led);
        input    [ TBD :0] sw;
        output [ TBD  :0] led;

        assign led = sw;
endmodule
```

```
#Constraint file, save as sw2led.xdc
set_property IOSTANDARD LVCMOS33 [get_ports {led[ ]}]
...
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[ ]}]
...
set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN F6 [get_ports {led[ ]}]
...
set_property PACKAGE_PIN K2 [get_ports {led[0]}]
set_property PACKAGE_PIN P5 [get_ports {sw[ ]}]
...
set_property PACKAGE_PIN R1 [get_ports {sw[0]}]
```

Practice-2



- To implement the logical relationship between inputs and outputs in this test(keep the state of the led remains at 8’b1100_0011 no matter how the dial switch changes, which of the following processes are needed ?

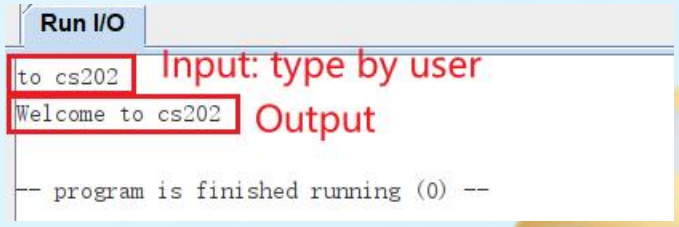
  1) update the design source file
  2) update the constraint source file
  3) update the testbench file
  4) regenerate the bitstream file
  5) re-program the device with the new bitstream file
  6) using a new FPGA chip to be programmed
  7) reset the chip type in the vivado project

A. 1,2,3,4,5,6,7 **B**. 6 **C**. 7 **D**. 1,2,3,4,5 **E**. 1,2,4,5 **F**. 3,4,5 **G**. 1,4,5 **H**. 2,4,5 **I**. 3,4,5 **J**. 1,4,5,6,7

TIPS1: Demo

- **Data declaration**
  - Data declaration section starts with "**. data**".
  - The declaration means **a piece of memory is required to be allocated**.
  - The declaration usually includes **lable** (name of address on this meomory unit), **type**, **size**(optional), and **initial value**(optional).

- **Code definition**
  - Code definition starts with "**.text**", includes **basic instructions, extended instructions, labels of the code**(optional).
  - At the end of the code, "**exit**" system service should be called.

li: a pseudo code that load immediate value into register
la: a pseudo code that register to label's address

```
# Demo
.data
        str:          .ascii   "Welcome "
        startpoint: .space  9

.text
main:
        li a7,8       # to get a string
        la a0,startpoint
        li a1,10
        ecall

        li a7,4       # to print a string
        la a0,str
        ecall

        li a7,10      # to exit
        ecall
```